

ROS2 四旋翼仿真项目

执行、测试与架构手册

Markdown 配套文档的 LaTeX 汇总版

项目	ros2_quadrotor_sim
目标环境	Ubuntu 22.04 / ROS2 Humble
基线日期	2026-07-24
文档状态	初版，可按阶段持续勾选和更新

由项目当前源码、配置、构建与实测结果整理

目录

1 文档目的与使用方法	1
2 当前项目基线	1
2.1 已确认实现	1
2.2 实测结果	1
3 当前系统架构	2
3.1 包级职责	2
3.2 控制闭环	3
3.3 运行时节点	3
3.4 ROS2 接口	3
3.5 电机顺序、坐标系与单位	4
3.6 动力学和控制器摘要	5
4 分阶段执行与测试计划	5
4.1 阶段总览	5
4.2 阶段 0: 闭环故障与验收可信度	6
4.2.1 工作	6
4.2.2 测试	7
4.3 阶段 1: 工程结构、元数据和脚本	7
4.3.1 工作	7
4.3.2 测试	7
4.4 阶段 2: 动力学模型验证	8
4.4.1 工作	8
4.4.2 测试	8
4.5 阶段 3: 控制器与 mixer 验证	8
4.5.1 工作	8
4.5.2 测试	9
4.6 阶段 4: 接口、TF、URDF 与 RViz	9
4.6.1 工作	9

4.6.2	测试	10
4.7	阶段 5: 场景测试与数据产品	10
4.7.1	工作	10
4.7.2	必做场景	10
4.7.3	每场景判据	11
4.8	阶段 6: README、技术文档和 AI 使用说明	11
4.9	阶段 7: 静态地图 (选做)	11
4.10	阶段 8: 路径规划与避障 (选做)	11
4.11	阶段 9: 回归与发布候选	12
4.12	阶段 10: 报告、PDF、视频和发布	12
5	项目级 Agent 规则摘要	12
5.1	稳定技术栈	12
5.2	事实优先级	13
5.3	代码与测试规则	13
5.4	稳定陷阱	13
6	Word 与报告写作规则	14
6.1	无模板回退页面样式	14
6.2	图表规则	14
6.3	数据与引用	15
6.4	模板加入后的处理	15
7	最终交付门禁	15
7.1	仓库	15
7.2	测试	16
7.3	报告与 PDF	16
7.4	视频	16
A	常用命令	16
B	阶段证据记录模板	17

1 文档目的与使用方法

本手册用于指导项目从当前核心仿真原型推进到可公开复现的最终交付。工作和测试项均带方框；只有在完成工作并留下证据后，才将 ☐ 改为 ☒。

- ☒：当前提交中已经核对，或已在本环境实际执行。
- ☐：尚未完成，或虽有代码但未按本手册重新验证。
- 每个阶段必须满足“阶段出口”后才能进入下一阶段。
- 历史交接记录只作为线索，不自动等同于当前结果。
- 可提交证据建议放在 `docs/evidence/phase-XX/`。

Word 模板状态

当前仓库和可见用户目录中没有发现 Word 模板文件。因此本文档中的字体、字号和页边距采用明确标注的“无模板回退样式”，不是从不存在的模板推断得出。正式模板到位后必须重新提炼样式。

2 当前项目基线

2.1 已确认实现

- ☒ `drone_dynamics` 已实现四电机一阶响应、6DoF 刚体动力学、阻力、地面约束、Odom、IMU、Path、实际 RPM 和 TF。
- ☒ `drone_controller` 已实现位置 PD、期望姿态、几何姿态反馈、角速度反馈、mixer 和限幅。
- ☒ `drone_bringup` 已包含参数、launch、URDF、RViz、目标 Marker 和单目标验收节点。
- ☒ `drone_map`、`drone_planner`、`drone_msgs` 当前仅为空包骨架。
- ☒ 仓库目前缺少正式 README、LICENSE、AI 使用说明、运行脚本、报告和最终演示材料。

2.2 实测结果

检查项	状态	结论
全包构建	通过	6 个 ROS2 包完成 colcon build --symlink-install。
现有自动测试	通过但覆盖不足	13 项、0 error、0 failure、3 skipped；主要是 Python 风格和 CMake/XML lint。
动力学/控制器单元测试	缺失	当前没有行为级数学测试。
默认系统验收	未通过	两次实测分别出现无 odometry、状态保持零且未收敛。
外层退出码	有误判风险	验收子进程 FAIL 时，外层 launch 仍可能返回 0。

当前最高优先级

先修复并稳定验证 goal -> controller -> motor RPM -> dynamics -> odom -> controller 闭环，同时修复失败退出码传播。不得在闭环未稳定前开始地图、规划或报告美化。

3 当前系统架构

3.1 包级职责

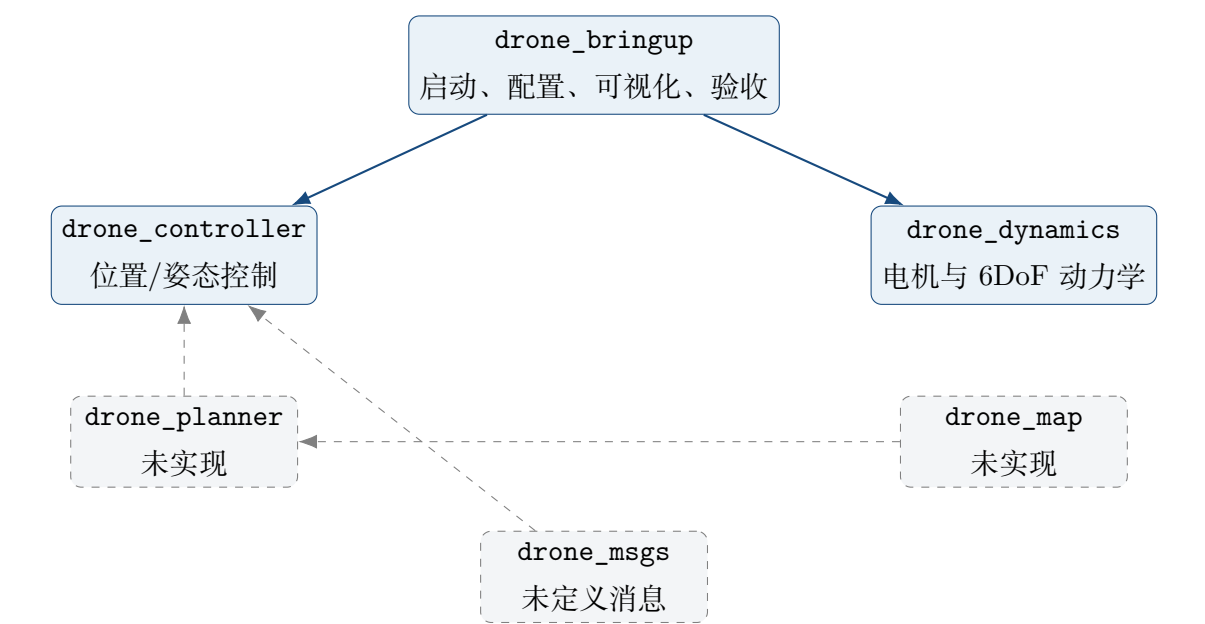


图 1: 当前包级架构；虚线框为尚未实现的可选能力

3.2 控制闭环

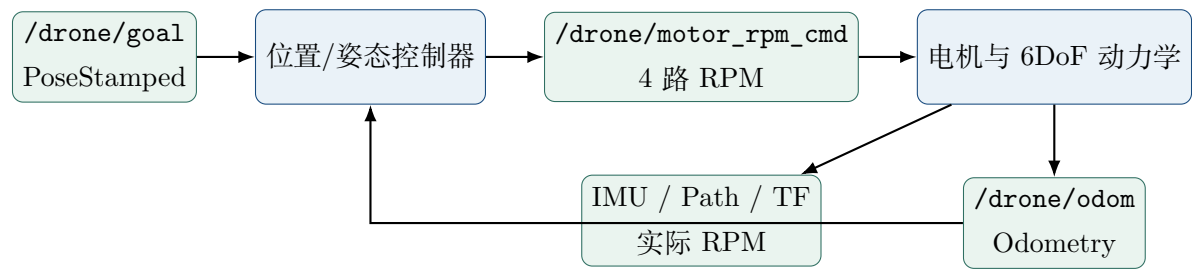


图 2: 当前核心控制闭环

3.3 运行时节点

节点	包	语言	频率	职责
quadrotor_dynamics_node	drone_dynamics	C++	积分 200 Hz	电机、动力学和状态发布
position_controller_node	drone_controller	C++	100 Hz	位置/姿态控制和 mixer
goal_marker_node	drone_bringup	Python	5 Hz	持续发布目标球
acceptance_test_node	drone_bringup	Python	10 Hz	发目标并判定稳定
robot_state_publisher	ROS2 外部包	C++	事件驱动	发布 URDF 固定变换
rviz2	ROS2 外部包	C++	30 FPS	显示模型、目标和轨迹

3.4 ROS2 接口

Topic	Type	Publisher	Subscriber / 用途
/drone/goal	geometry_msgs/PoseStamped	用户/验收	控制器、目标 Marker
/drone/motor_rpm_cmd	std_msgs/Float32MultiArray	控制器	动力学；4 路 RPM

Topic	Type	Publisher	Subscriber / 用途
/drone/motor_rpm	std_msgs/ Float32MultiArray	动力学	实际电机 RPM
/drone/odom	nav_msgs/Odometry	动力学	控制器、验收
/drone/imu	sensor_msgs/Imu	动力学	理想 IMU
/drone/path	nav_msgs/Path	动力学	RViz 实际轨迹
/drone/desired_ pose	geometry_msgs/ PoseStamped	控制器	RViz 期望姿态
/drone/goal_marker	visualization_msgs/ Marker	Marker 节点	RViz 目标球
/tf	tf2_msgs/TFMessage	动力学等	map->base_link
/tf_static	tf2_msgs/TFMessage	RSP	base_link->body_ link

3.5 电机顺序、坐标系与单位

电机顺序是硬接口：M1 前左、M2 前右、M3 后右、M4 后左。外部 topic 使用 RPM；进入物理模型前转换为 rad/s：

$$\omega_i = \text{rpm}_i \frac{2\pi}{60}, \quad (1)$$

$$F_i = k_F \omega_i^2. \quad (2)$$

当前力矩定义为：

$$\tau_x = \frac{l}{\sqrt{2}}(F_1 - F_2 - F_3 + F_4), \quad (3)$$

$$\tau_y = \frac{l}{\sqrt{2}}(-F_1 - F_2 + F_3 + F_4), \quad (4)$$

$$\tau_z = k_M(-\omega_1^2 + \omega_2^2 - \omega_3^2 + \omega_4^2). \quad (5)$$

TF 主链为：

$$\text{map} \longrightarrow \text{base_link} \longrightarrow \text{body_link}.$$

map -> base_link 为动力学动态变换；base_link -> body_link 为 URDF 固定变换。

3.6 动力学和控制器摘要

电机一阶响应:

$$\alpha = \text{clamp}\left(\frac{\Delta t}{\tau_m}, 0, 1\right), \quad (6)$$

$$\text{rpm}_a \leftarrow \text{rpm}_a + \alpha(\text{rpm}_c - \text{rpm}_a). \quad (7)$$

平动方程:

$$\mathbf{F}_w = \mathbf{R}_{bw} \begin{bmatrix} 0 & 0 & \sum_i F_i \end{bmatrix}^T, \quad (8)$$

$$\mathbf{a} = \frac{\mathbf{F}_w \begin{bmatrix} 0 & 0 & -mg \end{bmatrix}^T \mathbf{F}_{drag}}{m}. \quad (9)$$

转动方程:

$$\mathbf{I}\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}) - \mathbf{D}_\omega \boldsymbol{\omega}.$$

位置环:

$$\mathbf{a}_{des} = \mathbf{K}_p \odot (\mathbf{p}_{des} - \mathbf{p}) - \mathbf{K}_d \odot \mathbf{v}.$$

姿态环:

$$\mathbf{e}_R = \text{vee}\left(\frac{1}{2}(\mathbf{R}_d^T \mathbf{R} - \mathbf{R}^T \mathbf{R}_d)\right), \quad (10)$$

$$\boldsymbol{\tau} = -\mathbf{K}_R \odot \mathbf{e}_R - \mathbf{K}_\omega \odot \boldsymbol{\omega} + \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}). \quad (11)$$

4 分阶段执行与测试计划

4.1 阶段总览

阶段	范围	属性	阶段出口
0	基线与闭环故障处理	必做	默认验收连续 3 次 PASS, 失败返回非零
1	工程、元数据和脚本	必做	干净环境可重复构建和运行
2	动力学验证	必做	公式、符号、边界和频率有自动测试
3	控制器和 mixer 验证	必做	每个数学子步骤有测试
4	ROS 接口、TF、URDF、RViz	必做	接口契约和显示一致
5	场景测试与指标	必做	悬停、目标、多目标可复现
6	README 和技术文档	必做	新用户按文档可复现
7	静态地图	选做	至少 5 个障碍物可复现
8	规划与避障	选做	无碰撞到达且安全距离达标
9	回归与发布候选	必做	干净克隆全链路通过
10	报告、PDF、视频、发布	必做	最终交付物完整一致

4.2 阶段 0：闭环故障与验收可信度

4.2.1 工作

- ☒ 完成源码、配置、构建产物和历史材料盘点。
- ☒ 执行全包构建和现有测试。
- ☒ 重复执行默认验收并保留失败事实。
- ☐ 记录 ROS、RMW、Domain ID、localhost 和 WSL 网络环境。
- ☐ 用 `ros2 topic info -v` 核对 publisher、subscriber 和 QoS。
- ☐ 分别检查 odom、RPM 命令和实际 RPM 是否持续更新。
- ☐ 区分代码、定时器、执行器与 DDS 发现问题。
- ☐ 修复导致闭环不运动或消息偶发断开的根因。
- ☐ 让验收子进程失败可靠传播为外层非零退出码。
- ☐ 冻结基线参数并记录参数文件 SHA-256。

4.2.2 测试

- ☐ 冷启动 3 秒内出现 odom, 频率约 50 Hz。
- ☐ 收到目标后 RPM 命令约 100 Hz。
- ☐ 实际 RPM 能跟踪命令并体现电机一阶动态。
- ☐ 默认目标连续 3 次 PASS。
- ☐ 极短超时、停止控制器、错误 topic 均明确 FAIL。
- ☐ PASS、FAIL、Ctrl+C 三条退出路径均无残留进程。

阶段 0 出口

- ☐ 默认验收连续 3 次 PASS; ☐ 故障注入稳定 FAIL; ☐ 退出码与结果一致;
- ☐ 根因和证据已归档。

4.3 阶段 1：工程结构、元数据和脚本

4.3.1 工作

- ☐ 创建 README、LICENSE 和 AI 使用说明骨架。
- ☐ 替换包描述、版本、维护者和邮箱中的模板占位信息。
- ☐ 核对 package.xml、CMake 和源码实际依赖。
- ☐ 所有使用 C++17 的包显式设置 C++17。
- ☐ 决定空包保留策略，并在 README 中如实标记。
- ☐ 创建 build、clean、run、acceptance、topic-check 和 test 脚本。
- ☐ 脚本统一使用 `set -euo pipefail` 并自动定位根目录。
- ☐ 处理错误的 Zone.Identifier，完善.gitignore。

4.3.2 测试

- ☐ 删除 build/install/log 后从零构建成功。
- ☐ 脚本从任意工作目录调用仍能定位仓库。
- ☐ 无 RViz 模式不启动 GUI 且核心节点完整。
- ☐ clean 脚本只删除仓库内生成目录。

- ☐ rosdep 能解析所有声明依赖。
- ☐ 阶段 0 验收无回归。

4.4 阶段 2：动力学模型验证

4.4.1 工作

- ☐ 把纯数学计算从 ROS 回调提取为可测试函数。
- ☐ 明确状态、坐标系、旋向、符号、单位和积分顺序。
- ☐ 为质量、惯量、频率、系数、阻力和 RPM 上限增加校验。
- ☐ 明确 IMU 是理想比力，当前没有噪声、偏置和协方差。
- ☐ 明确使用墙钟；若新增 /clock，单独设计和测试。
- ☐ 将 Path 上限改为可配置或明确长期运行限制。

4.4.2 测试

- ☐ 零 RPM 时推力和力矩为零。
- ☐ 四电机相同 RPM 时三轴力矩接近零。
- ☐ 理论悬停 RPM 与 $\sqrt{mg/(4k_F)}$ 一致。
- ☐ 单电机扰动的 roll/pitch/yaw 符号正确。
- ☐ 阶跃响应符合电机时间常数。
- ☐ 负值、上溢、NaN、Inf 均被安全处理。
- ☐ 四元数长时间积分后保持归一化。
- ☐ 无推力自由落体、地面约束和阻力方向正确。
- ☐ 发布 frame、频率、实际 RPM 和 TF 正确。
- ☐ 运行 10 分钟无 NaN、状态重置或异常增长。

4.5 阶段 3：控制器与 mixer 验证

4.5.1 工作

- ☐ 提取位置环、限幅、期望姿态、姿态误差和 mixer 纯函数。

- ☐ 明确当前为 PD 控制，不宣称存在积分项。
- ☐ 对控制参数增加有限值、正值和范围校验。
- ☐ 校验控制器和动力学共享物理参数一致。
- ☐ 记录远目标截断、倾角限制和推力上限的真实语义。
- ☐ 定义电机饱和时的可接受退化行为。

4.5.2 测试

- ☐ 缺少 goal 或 odom 时四电机输出为零。
- ☐ 零误差水平姿态时输出理论悬停 RPM。
- ☐ x/y/z 误差产生正确方向的期望加速度。
- ☐ 水平、竖直加速度与倾角限幅正确。
- ☐ 目标四元数归一化，零四元数按 yaw=0 处理。
- ☐ 超过 10 m 的目标按定义截断。
- ☐ 几何姿态误差和 torque 符号正确。
- ☐ mixer 正反算一致，输出始终有限并在范围内。
- ☐ 悬停、纯轴向、纯 yaw 和大阶跃场景不发散。

4.6 阶段 4：接口、TF、URDF 与 RViz

4.6.1 工作

- ☐ 冻结 topic、type、QoS、publisher、subscriber、rate 和 frame。
- ☐ 评估是否继续使用 Float32MultiArray；改消息时提供迁移说明。
- ☐ 统一 map -> base_link -> body_link 语义。
- ☐ 核对 URDF 质量/惯量与 YAML 和动力学一致。
- ☐ RViz 显示模型、轨迹、目标、期望姿态和必要 TF。
- ☐ 为 topic 和 TF 添加机器可检查的契约测试。

4.6.2 测试

- ☐ 运行时 topic 类型与架构表完全一致。
- ☐ 每个 topic 的发布者、订阅者和 QoS 符合契约。
- ☐ TF 无断链、重复发布或环。
- ☐ RobotModel 姿态、机头方向和实际运动一致。
- ☐ 目标 Marker、Desired Pose 和 Path 与消息一致。
- ☐ `rviz:=false` 不改变核心仿真。

4.7 阶段 5：场景测试与数据产品

4.7.1 工作

- ☐ 把单目标验收纳入 colcon 可管理的集成测试。
- ☐ 实现 3-4 个 waypoint 的多目标场景运行器。
- ☐ 记录位置、速度、姿态、目标、四电机 RPM 和误差。
- ☐ 输出 CSV/JSON 原始数据和可重复绘图脚本。
- ☐ 定义到达时间、超调、稳态误差、饱和占比和路径长度。
- ☐ 随机或扰动场景固定种子。

4.7.2 必做场景

- ☐ 悬停 $(0, 0, 1.5, 0^\circ)$ 。
- ☐ 单目标 $(2, 1, 1.5, 90^\circ)$ 。
- ☐ 扩展目标 $(3, -1, 2, 45^\circ)$ 。
- ☐ 负坐标目标 $(-1, 2, 1, -90^\circ)$ 。
- ☐ 正方形或矩形 4 waypoint。
- ☐ 到达后稳定保持至少 10 秒。
- ☐ 关键场景各重复至少 3 次。

4.7.3 每场景判据

- ☐ 位置、偏航、线速度、角速度和稳定时间均达标。
- ☐ 无 NaN、重置、崩溃、TF 中断和长期 RPM 饱和。
- ☐ 保存到达时间、最大超调、稳态误差和 RPM 曲线。
- ☐ 任务最低 0.3 m 与项目严格 0.05 m 标准分开报告。

4.8 阶段 6：README、技术文档和 AI 使用说明

- ☐ README 覆盖简介、架构、依赖、构建、运行、目标、验收、接口、参数、结果和限制。
- ☐ 完成动力学、控制器、ROS 接口和测试文档。
- ☐ `ai_usage.md` 记录至少 8 条关键 prompt/交互摘要。
- ☐ 记录 AI 错误、发现方式和修复证据。
- ☐ 说明参考项目中参考、重写和差异化的内容。
- ☐ 在干净环境中逐条执行 README 命令。
- ☐ 核对文档公式、默认值、接口和结果与代码一致。

4.9 阶段 7：静态地图（选做）

- ☐ 实现地图节点和可计算的障碍物数据。
- ☐ 提供至少 5 个起终点之间的静态障碍物。
- ☐ 使用固定场景或固定随机种子。
- ☐ 定义边界、无人机半径和安全膨胀距离。
- ☐ 发布 `/map/obstacles` 并在 RViz 显示。
- ☐ 验证同一配置可复现、起终点合法、直线受阻。
- ☐ 验证 Marker 几何与碰撞几何一致。

4.10 阶段 8：路径规划与避障（选做）

- ☐ 先写算法输入、输出、假设和失败条件。
- ☐ 优先实现直线碰撞检查加绕行 waypoint，或 3D 栅格 A*。

- ☐ 分离用户最终目标和控制器安全参考 topic。
- ☐ 对障碍物按机体半径和安全距离膨胀。
- ☐ 显示规划路径、当前局部目标和实际轨迹。
- ☐ 覆盖无障碍、单障碍、多障碍、狭窄通道、无解和非法目标。
- ☐ 实际轨迹最小障碍物距离必须大于安全距离。

4.11 阶段 9：回归与发布候选

- ☐ 冻结依赖、默认参数、接口和场景。
- ☐ 建立 build、lint、unit、integration、scenario 总回归命令。
- ☐ 覆盖非法参数、非法消息、节点晚启动和异常退出。
- ☐ 运行 10 分钟和 30 分钟资源稳定性测试。
- ☐ 检查高频日志、残留进程、DDS 参与者和内存增长。
- ☐ 干净克隆后完成 rosdep、构建、测试和验收。
- ☐ 最终 Git 状态不包含生成物或敏感信息。

4.12 阶段 10：报告、PDF、视频和发布

- ☐ 按本手册文档规范完成 6–10 页报告。
- ☐ 从最终测试数据重新生成全部图表。
- ☐ 完成架构图、动力学流程、控制流程和 TF 图。
- ☐ 用 LaTeX 源文件编译 PDF，并记录可复现命令。
- ☐ 逐页检查 PDF 的缺字、重叠、裁切和图表清晰度。
- ☐ 录制 1–3 分钟视频，展示启动、RViz、轨迹、目标和验收。
- ☐ 推送公开仓库并创建对应最终提交的版本标签。

5 项目级 Agent 规则摘要

5.1 稳定技术栈

- Ubuntu 22.04、ROS2 Humble、colcon、ament。

- 核心动力学和控制使用 C++17 与 Eigen3。
- Bringup、Marker 和测试编排使用 Python 3 与 rclpy。
- 参数使用 YAML，可视化使用 RViz2、URDF 和 tf2。

5.2 事实优先级

1. 当前提交和参数下实际执行的测试；
2. 源码和 launch；
3. `vertical_sim.yaml`；
4. 当前文档；
5. 历史交接记录。

5.3 代码与测试规则

- 变更前检查 Git，保留无关用户修改。
- 只修改最小必要范围；算法、格式、文档和新功能不要混成一个提交。
- 数学逻辑优先提取为无 ROS 依赖的可测试函数。
- 外部数值输入检查有限性，启动参数检查物理合法性。
- 修改动力学、控制器、参数、topic、TF、QoS 或 launch 后必须跑核心验收。
- 接口变更、跨包变更、规划集成和发布前必须跑全量回归。
- 验收成功必须同时检查显式 PASS、测试进程结果和外层结果。
- 不能通过无依据放宽阈值来隐藏失败。

5.4 稳定陷阱

- 动力学与控制器重复声明物理参数，配置漂移不会触发编译错误。
- topic 使用 RPM，推力和反扭矩系数使用 rad/s 的平方。
- mixer 与动力学电机符号耦合，局部改符号会反转姿态轴。
- Float32MultiArray 没有结构信息，必须验证长度并固定电机顺序。
- 目标四元数为零时按 yaw=0 处理。
- goal 或 odom 未就绪时控制器输出零 RPM。

- 动态 TF 与固定 TF 来源不同，重复发布会产生冲突。
- 当前使用墙钟，不能未经设计就假设支持仿真时间。
- IMU 是理想模型，不具备真实传感器噪声和协方差。
- 独立电机饱和会改变期望力/力矩比例。
- 空包可构建不代表功能已实现。
- build 和 lint 通过不代表飞行闭环可用。

6 Word 与报告写作规则

6.1 无模板回退页面样式

项目	规则
纸张	A4 纵向，单栏
页边距	上下左右均 25.4 mm
页码	页脚居中，阿拉伯数字
页眉	默认不使用；正式模板优先
分页	使用分页符/分节符，不用空行控制
正文	中文宋体 12 pt，英文和数字 Times New Roman 12 pt，1.5 倍行距
一级标题	黑体 16 pt，加粗
二级标题	黑体 14 pt，加粗
三级标题	黑体 12 pt，加粗
图表题	宋体 10.5 pt，英文和数字 Times New Roman
代码	Consolas 9.5 pt
公式	Cambria Math 11 pt

6.2 图表规则

- 图题在图下，表题在表上；使用自动题注和交叉引用。
- 优先 SVG/EMF；位图建议 300 ppi，最低 150 ppi。

- 图表必须有标题、轴名、单位、图例和场景说明。
- 参数表包含名称、符号、值、单位和来源。
- 结果表包含场景、提交、参数集、运行次数、指标和 PASS/FAIL。
- 不使用三维柱图装饰简单数据，不截断坐标轴而不说明。
- 不平滑原始数据而不说明滤波方法。
- 不用代码或公式截图替代可编辑文本。

6.3 数据与引用

正式实验必须绑定 Git commit、参数文件 SHA-256、ROS2 版本、RMW、运行命令、原始数据和绘图脚本。历史指标必须复测后才能写为当前结果。至少保留一个有解释价值的失败案例。

6.4 模板加入后的处理

- ☐ 记录模板路径、SHA-256、页数和分节数。
- ☐ 逐页渲染并检查封面、正文、表格、页眉和页脚。
- ☐ 提取纸张、边距、标题层级、正文、题注、列表和表格实测值。
- ☐ 检查字段、自动编号、交叉引用、文本框、内容控件、批注和修订。
- ☐ 用模板实测值替换回退样式，并生成样章比对。

7 最终交付门禁

7.1 仓库

- ☐ 公开 Git 仓库可访问。
- ☐ README 可指导新用户完成构建和运行。
- ☐ build/install/log 和临时文件未提交。
- ☐ 最终标签对应全量回归通过的提交。

7.2 测试

- ☐ 全包构建通过。
- ☐ lint、单元、集成和场景测试通过。
- ☐ 默认验收连续 3 次 PASS。
- ☐ 故障注入可靠 FAIL 并返回非零。
- ☐ 结果数字可追溯到原始数据。

7.3 报告与 PDF

- ☐ 正文覆盖架构、动力学、控制器、接口、实验、AI 使用和反思。
- ☐ 图、表、公式编号和目录正确。
- ☐ PDF 逐页视觉检查，无缺字、重叠、裁切和模糊。
- ☐ PDF 参数、topic 和测试数字与最终代码一致。
- ☐ LaTeX 可从记录命令重新编译。

7.4 视频

- ☐ 时长 1-3 分钟。
- ☐ 显示启动命令、RViz 无人机、目标、轨迹和验收结果。
- ☐ 使用最终发布提交，不使用旧版本录屏。

A 常用命令

```
cd ~/ros2_quadrotor_sim
source /opt/ros/humble/setup.bash

colcon build --symlink-install
source install/setup.bash

colcon test
colcon test-result --verbose

ros2 launch drone_bringup sim.launch.py
```

```
ros2 launch drone_bringup sim.launch.py rviz:=false
ros2 launch drone_bringup acceptance_test.launch.py
```

在验收退出码链路修复并测试前，成功判据必须同时包含：

```
ACCEPTANCE TEST: PASS
acceptance_test_node exit code 0
wrapper or CI exit code 0
```

B 阶段证据记录模板

```
# Phase XX 验证记录

- Commit:
- ROS distro:
- RMW implementation:
- Parameter file SHA-256:
- Date:

## Commands
## Work completed
## Test results
## Metrics
## Failures and root causes
## Artifacts
## Phase gate decision
```

配套源文档

完整 Markdown 清单见：

- docs/PROJECT_EXECUTION_PLAN.md
- docs/ARCHITECTURE_OVERVIEW.md
- AGENTS.md
- docs/WORD_WRITING_GUIDE.md

LaTeX 本文档为便于打印、审阅和导出 PDF 的汇总版；阶段推进时，以 Markdown 清单作为可直接勾选的主记录。